

Analisis & Desain Algoritma Sorting Lanjutan

a. Ruang & Distribusi Sort

Implementasi standar Radix Sort menggunakan array berisi 10 queue untuk menyimpan digit 0–9 pada setiap proses distribusi data. Setiap elemen sementara dipindahkan ke bucket sesuai digitnya sebelum dikembalikan ke array utama. Akibatnya, dibutuhkan ruang tambahan yang cukup besar karena semua elemen harus disimpan sementara selama proses sorting. Oleh karena itu, penggunaan memori tidak memenuhi batasan $O(1)$ pada Modul A dan lebih mendekati $O(n)$.

Improved Merge Sort menggunakan satu tmpArray global yang dibuat sekali di awal proses sorting. Berbeda dengan pendekatan berbasis slice yang membuat sub-array baru di setiap rekursi, teknik virtual sublists hanya menggunakan indeks untuk membagi array tanpa menyalin data. Dengan demikian, penggunaan memori menjadi lebih efisien karena array sementara dipakai ulang selama proses merge berlangsung.

Jika tetap ingin memanfaatkan distribusi digit seperti Radix Sort tanpa melanggar batasan $O(1)$, strategi yang paling realistis adalah menggunakan in-place radix variation seperti American Flag Sort. Algoritma ini mendistribusikan elemen langsung di dalam array tanpa bucket besar tambahan. Namun implementasinya lebih kompleks dan membutuhkan banyak operasi swap sehingga waktu eksekusinya sedikit lebih lambat dibanding Radix Sort biasa.

b. Linked List & Pointer Manipulation

Fungsi `_splitLinkedList()` menggunakan teknik fast-slow pointer untuk menemukan titik tengah linked list hanya dalam satu traversal. Pointer `midPoint` bergerak satu langkah sedangkan `curNode` bergerak dua langkah sekaligus. Ketika `curNode` mencapai akhir linked list, `midPoint` otomatis berada di tengah list. Teknik ini lebih efisien karena tidak perlu menghitung panjang linked list terlebih dahulu.

Pada proses merge, fungsi `_mergeLinkedLists()` menggunakan dummy node dan tail reference untuk menghubungkan node lama tanpa membuat node baru. Pointer `.next` hanya dipindahkan sesuai urutan data yang lebih kecil. Karena node yang digunakan tetap node lama, penggunaan memori tambahan hampir tidak ada. Kompleksitas ruang hanya berasal dari stack rekursi merge sort yaitu $O(\log n)$.

c. Quick Sort Worst-Case & Pivot Strategy

Jika Quick Sort dijalankan pada data descending, pivot dapat terus menjadi elemen terkecil atau terbesar. Hal ini membuat partisi menjadi sangat tidak seimbang karena satu sisi hampir berisi seluruh elemen sedangkan sisi lain kosong. Pointer left dan right bergerak tidak efisien sehingga kedalaman rekursi meningkat hingga $O(n)$. Akibatnya, kompleksitas waktu memburuk menjadi $O(n^2)$.

Strategi yang lebih baik adalah median-of-three pivot, yaitu memilih median dari elemen pertama, tengah, dan terakhir. Strategi ini membantu menghasilkan partisi yang lebih seimbang dan mengurangi kemungkinan worst-case. Namun pada singly linked list strategi ini kurang efisien karena akses elemen tengah membutuhkan traversal $O(n)$, berbeda dengan array yang mendukung akses langsung.

d. Batas Teoretis & Paradigma Sorting

Radix Sort tidak melanggar batas bawah $\Omega(n \log n)$ karena algoritma ini bukan comparison sort. Radix Sort tidak membandingkan elemen satu per satu seperti Merge Sort atau Quick Sort, melainkan mengelompokkan data berdasarkan digit atau karakter tertentu. Ada dua asumsi penting agar Radix Sort dapat mencapai kompleksitas $O(dn)$. Pertama, jumlah digit (d) harus relatif kecil dan tetap. Kedua, rentang digit (k) juga kecil, misalnya hanya 0–9. Jika nilai d atau k sangat besar, maka efisiensi Radix Sort akan menurun baik dari sisi waktu maupun memori.